demo-猴哥成语大冲关-前端开发文档

一、项目概述

这是一款成语答题 + 每日冲关 + 月度身份晋升的小游戏。核心玩法：玩家需要将打乱顺序的4个成语按正确顺序点击排列，在限定时间内连续答对5题通过一局，连续通过5局完成当日冲关。游戏包含时间双倍（广告增益）、跳过、复活（广告增益）等辅助功能，以及基于通关天数的月度身份晋升系统。

本版本文档基于 Supabase + GitHub 方案重新设计，替代原有的抖音云 + 抖音小游戏 API 架构。核心目标是先跑通功能 Demo，确保基本游戏流程可用，后续再逐步对接抖音小游戏平台的原生 API。

前端外包团队负责范围：

- 所有前端页面的搭建、动效实现
- 广告 SDK 对接（Demo 阶段使用 Mock，后期切换抖音激励视频广告）
- Supabase Client SDK 集成（认证、数据库查询、Edge Functions 调用）
- 数据上报接口调用
- 不负责任何业务判定逻辑（如判断答对/答错、是否冲关成功，这些均由后端接口返回）

二、技术规范与要求

2.1 技术栈

类别	技术选型	说明
框架	React 18+	函数组件 + Hooks
样式	TailwindCSS	优先使用内置类名
语言	TypeScript	严格类型检查
构建工具	Vite	快速开发与构建
路由	react-router-dom	单页应用路由
状态管理	React Hooks	useReducer / useContext，GameStatus 枚举
后端服务	Supabase	@supabase/supabase-js 客户端 SDK
认证	Supabase Auth	匿名登录 + 手机号登录

数据库	Supabase PostgreSQL	通过 Client SDK 直接查询（配合 RLS）
业务逻辑	Supabase Edge Functions	Deno 运行时，复杂业务判定逻辑
文件存储	Supabase Storage	用户头像等文件存储

2.2 开发环境

开发阶段使用浏览器预览即可，无需依赖抖音小游戏开发者工具。构建时需将产物适配为抖音小游戏可加载的 Web 包（抖音小游戏支持 WebView 加载或通过 tt.loadUrl 加载 H5 页面）。Demo 阶段可直接作为标准 Web 应用运行，部署在 GitHub Pages 或 Vercel 上。

2.3 目录结构

```
chengyu-game-frontend/
├── package.json
├── tsconfig.json
├── tailwind.config.js      # TailwindCSS 配置
├── postcss.config.js       # PostCSS 配置
├── vite.config.ts          # Vite 构建配置
├── .env.local              # Supabase 环境变量
├── index.html              # 入口 HTML
├── public/
│   ├── images/
│   ├── audio/
│   └── fonts/
├── src/
│   ├── types/
│   │   └── types.ts        # 【核心】类型定义文件
│   ├── config/
│   │   ├── supabase.ts     # Supabase Client 初始化
│   │   ├── adConfig.ts     # 广告位 ID 配置
│   │   └── apiConfig.ts    # 其他配置
│   ├── lib/
│   │   ├── supabase.ts     # Supabase 封装（auth/db/functions/storage）
```

```
│   │   └── adMock.ts          # 广告 Mock 封装（Demo用）
│   ├── api/
│   │   ├── user.ts            # 用户相关接口
│   │   ├── game.ts            # 游戏相关接口
│   │   └── gift.ts            # 礼品接口（二期）
│   ├── hooks/
│   │   ├── useGame.ts         # 游戏核心状态管理 Hook
│   │   └── useAd.ts           # 广告 Hook（Demo: Mock）
│   ├── components/            # 通用 UI 组件
│   │   ├── AnswerButton.tsx   # 可点击的成语按钮
│   │   ├── CountdownTimer.tsx # 倒计时组件
│   │   ├── ProgressDots.tsx   # 进度指示点
│   │   ├── ModalBase.tsx      # 弹窗基类
│   │   └── IdentityBadge.tsx  # 身份等级徽章
│   ├── pages/                 # 页面组件
│   │   ├── LaunchPage.tsx     # 启动页
│   │   ├── HomePage.tsx       # 首页
│   │   ├── GamePage.tsx       # 游戏主页面
│   │   ├── ProfilePage.tsx    # 个人中心
│   │   └── GiftPage.tsx       # 礼品页（二期）
│   ├── router/
│   │   └── index.tsx          # 路由配置
│   ├── App.tsx
│   ├── main.tsx
│   └── styles/
│       └── index.css          # TailwindCSS 指令 + 全局样式
└── README.md
```

2.4 代码质量要求

- 所有组件、函数需有清晰注释说明用途
- 跨组件通信使用事件总线 EventBus，避免硬耦合

- src/types/types.ts 必须严格按照本文档附件中的类型定义编写，不得自行增删改字段
- Supabase 环境变量统一放在 .env.local 中，不得硬编在代码里

三、Supabase 集成说明

本项目用 Supabase 替代抖音云作为后端服务，前端通过 @supabase/supabase-js 客户端 SDK 与 Supabase 交互。以下是各模块的集成方式说明。

3.1 Supabase Client 初始化

在 src/config/supabase.ts 中初始化 Supabase 客户端，从环境变量读取 Supabase URL 和 Anon Key。Anon Key 是公开的前端安全密钥，配合 Row Level Security (RLS) 策略保证数据安全。所有数据库查询均通过该客户端实例进行，确保请求自动携带用户认证信息。

```
// src/config/supabase.ts

import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(

  import.meta.env.VITE_SUPABASE_URL,

  import.meta.env.VITE_SUPABASE_ANON_KEY

)
```

3.2 认证方案

原抖音小游戏方案使用 tt.login() 获取 openid 进行用户识别，现改为 Supabase Auth 方案。Demo 阶段使用匿名登录，无需用户手动操作，进入游戏时自动创建匿名用户。后续对接抖音小游戏时，可通过 Supabase Auth 的自定义 Provider 或手机号登录进行用户绑定。

认证方式	使用场景	说明
匿名登录	Demo 阶段 / 首次进入	supabase.auth.signInAnonymously()，自动生成 UUID 作为用户标识
手机号登录	二期 / 绑定手机	supabase.auth.signInWithOtp()，验证码登录
抖音登录	上架抖音后	通过自定义 Provider 或将 openid 关联到 Supabase 用户

3.3 数据库访问

简单查询（如获取用户信息、查询今日冲关进度等）可通过 Supabase Client SDK 直接查询 PostgreSQL 数据库，配合 Row Level Security (RLS) 策略确保用户只能访问自己的数据。复杂业务逻

辑（如开始新局、提交答案、单局结束等）通过 Supabase Edge Functions 实现，前端通过 supabase.functions.invoke() 调用。

```
// 直接查询示例：获取当前用户信息
const { data, error } = await supabase
  .from('users')
  .select('*')
  .eq('id', currentUserId)
  .single()

// Edge Function 调用示例：开始新局
const { data, error } = await supabase.functions.invoke('session-start', {
  body: { userId: currentUserId }
})
```

3.4 实时订阅（可选）

Supabase 提供 Realtime 实时订阅功能，可用于未来扩展的实时功能（如排行榜更新、全服广播等）。Demo 阶段暂不使用，但前端可预留接口。

3.5 文件存储

用户头像上传等文件操作通过 Supabase Storage 实现，替代原来的抖音云存储。使用 supabase.storage.from('avatars').upload() 上传文件，通过 getPublicUrl() 获取公开访问地址。

四、路由与页面结构

4.1 路由表

路径	组件	说明
/	LaunchPage	启动页（认证初始化）
/home	HomePage	首页（展示身份/积分/进度）
/game	GamePage	游戏主页面
/profile	ProfilePage	个人中心
/gift	GiftPage	礼品页（二期占位）

4.2 页面跳转关系

启动页 → [非法用户] → 拒绝登录

启动页 → [正常/异常] → 首页

首页 → 开始游戏 → 游戏页

首页 → 个人中心 → 个人中心页

首页 → 礼品页 → 礼品页（二期）

游戏页 → 返回首页 → 首页

游戏页 → 单局结束 → 继续挑战/返回首页

游戏页 → 每日冲关结束 → 返回首页

五、页面功能详细说明

5.1 启动页 (LaunchPage)

启动页是用户进入游戏的第一个页面，主要完成用户认证初始化工作。原抖音方案通过调用后端 POST /api/user/init 接口，现改为通过 Supabase Auth 完成匿名登录，然后调用 Edge Function user-init 初始化用户数据。如果用户已存在，则直接获取其完整信息（积分、身份、风控状态、今日进度）。

核心逻辑：

- 检查 Supabase Auth 会话，若未登录则调用 supabase.auth.signInAnonymously()
- 调用 Edge Function user-init 初始化用户数据（创建或更新 users 表记录）
- 展示加载动画和健康游戏忠告：“抵制不良游戏，拒绝盗版游戏。注意自我保护，谨防受骗上当。”
- 根据接口返回的 userStatus 字段：正常(0) 跳转 /home；异常(1) 跳转 /home（首页显示异常提示）；非法(2) 全屏显示“非法用户，拒绝登录！”

5.2 首页 (HomePage)

首页展示用户的核心信息和每日冲关进度。数据通过调用 Edge Function user-init 或直接查询 Supabase 数据库获取，前端不做任何业务逻辑计算。

需展示的数据：

- 用户身份等级文字+图标（书生、童生、秀才、举人、进士、状元、太傅、皇帝）
- 当前总积分
- 当日冲关进度（例如：已完成 3/5 局，用进度点或进度条表示）
- 异常用户需显示醒目文字提示：“你有异常行为，暂定增益功能和积分奖励！”

交互入口：

- “开始游戏”按钮 → 跳转 /game
- “个人中心”按钮 → 跳转 /profile
- “礼品页”按钮 → 跳转 /gift（二期）

5.3 游戏主页面 (GamePage)

5.3.1 界面元素

元素	说明
倒计时器	显示剩余秒数，超时触发游戏失败逻辑
当前题号 / 总题数	如“3/5”，表示第3题共5题
成语按钮区	4个可点击的成语按钮，打乱顺序展示
已选顺序区	展示用户已点选的成语顺序
跳过按钮	显示剩余跳过次数，耗尽后置灰
本局积分	显示当前局已获得积分
广告增益入口	时间双倍/复活按钮，根据状态显隐

5.3.2 核心游戏流程

以下是游戏核心流程，所有业务判定均由后端 Edge Functions 完成，前端只负责上报操作和展示结果。

- 1. 进入游戏页 → 调用 Edge Function session-start 获取新一局数据
- 2. 检查时间双倍 → canUseTimeDouble=true 时，弹出“时间双倍增益询问”弹窗
- 3. 开始答题 → 启动倒计时，展示第一道题
- 4. 玩家点选成语 → 界面实时更新已选顺序 → 4个成语全部选定后自动提交
- 5. 提交答案 → 调用 Edge Function session-answer，传入 { sessionId, questionId, answer }
- 6. 展示结果 → 根据 isCorrect 播放答对/答错动效，更新本局积分
- 7. 下一题 → 短暂展示后（约0.5秒）自动加载下一题
- 8. 点击跳过 → 调用 Edge Function session-skip，更换新题，跳过次数-1
- 9. 完成5题 → 调用 Edge Function session-end（isTimeout: false）

- 10. 倒计时归零 → 调用 Edge Function session-end（isTimeout: true），根据 canRevive 决定弹窗

5.3.3 前端状态管理

前端应维护 GameStatus 枚举（见附件 types.ts）。所有业务数据（积分、是否通关、是否可复活等）均以后端接口返回为准，前端不做本地逻辑判断。

5.4 个人中心 (ProfilePage)

需展示的数据：

- 头像、昵称（通过 Supabase Auth 获取）
- 手机号（若已通过 Supabase Auth 手机号登录绑定）
- 所在地区（若已授权定位，使用浏览器 Geolocation API）
- 贡献值评级（1-10）
- 身份等级、总积分
- 礼品领取历史记录（二期）

交互功能：

- 绑定手机号：调用 supabase.auth.signInWithOtp()，验证码登录并绑定
- 授权定位：调用浏览器 navigator.geolocation.getCurrentPosition()，将结果传给后端

注意：原抖音方案使用 tt.getPhoneNumber 和 tt.getLocation，现替换为通用 Web API。后续对接抖音小游戏平台时再切换回平台特有 API。

5.5 礼品页 (GiftPage)（二期，先占位）

- 展示当月可领取的礼品列表
- 每个礼品标注所需身份等级
- 点击领取，调用 Edge Function gift-claim
- 展示历史领取记录

六、弹窗与提示体系

所有弹窗只需明确展示内容、按钮文案和触发条件，视觉样式由前端团队自行设计（推荐使用统一的 ModalBase 组件）。以下是所有弹窗场景的完整列表：

弹窗名称	触发条件	展示内容	按钮
时间双倍询问			“观看广告” / “不需要”

	canUseTimeDouble=true 且局序号≤5	“是否观看广告获得时间双倍？”	
广告播放中	点击观看广告	广告视频播放中（Demo: Mock 延迟）	无按钮，自动关闭
单局通过	答对5题 isSessionPassed=true	“恭喜通过！” + 本局积分	“继续挑战” / “返回首页”
单局失败（可复活）	isSessionPassed=false 且 canRevive=true	“挑战失败，是否观看广告复活？”	“观看广告复活” / “放弃”
单局失败（不可复活）	isSessionPassed=false 且 canRevive=false	“挑战失败！” + 本局积分	“返回首页”
每日冲关成功	dailyPassed=true	“今日冲关成功！” + 积分汇总	“返回首页”
异常用户提示	异常用户点击增益功能	“你有异常行为，暂定增益功能和积分奖励！”	“知道了”
达到每日上限	maxSessionsReached=true	“今日已达最大局数，明日再来！”	“返回首页”

七、接口清单与调用规则

7.1 统一规则

- 所有接口通过 Supabase Edge Functions 或直接数据库查询实现
- Edge Functions 返回格式统一为 ApiResponse<T>（见附件 types.ts）
- 请求自动携带 Supabase Auth JWT 凭证，由 SDK 处理，前端无需手动传 token
- src/api/ 下各文件只负责请求发送和响应解析，不做业务逻辑处理

7.2 接口列表

接口名称	调用方式	说明
user-init	Edge Function	用户初始化/登录，创建或获取用户完整信息
session-start	Edge Function	开始新一局，返回 sessionId、题目、倒计时、跳过次数
session-answer	Edge Function	提交答案，返回 isCorrect、pointsEarned
session-skip	Edge Function	跳过当前题，返回新题目

session-end	Edge Function	单局结束，返回 isSessionPassed、canRevive、dailyStatus
time-double-use	Edge Function	使用时间双倍，返回新倒计时
revive-confirm	Edge Function	复活确认，返回新局数据
daily-status	直接查询	查询今日冲关进度（supabase.from 查询）
gift-claim	Edge Function	礼品领取（二期）

7.3 接口调用示例

```
// 调用 Edge Function
const { data, error } = await supabase.functions.invoke('session-start', {
  body: { userId: user.id }
})

// 直接查询数据库
const { data, error } = await supabase
  .from('daily_challenge')
  .select('*')
  .eq('user_id', user.id)
  .eq('challenge_date', today)
  .single()
```

八、广告系统交互规则

8.1 Demo 阶段：Mock 广告

由于本方案先基于 Web 跑通 Demo，无法使用抖音小游戏的 tt.createRewardedVideoAd 激励视频广告 API，因此 Demo 阶段采用 Mock 广告方案。Mock 广告将模拟观看广告的完整流程：弹出广告窗口 → 倒计时 5 秒 → 自动关闭 → 触发 onClose 回调（isEnded=true）。这样可以确保前端流程逻辑与正式广告一致，后续对接抖音广告 SDK 时只需替换 Hook 内部实现即可。

```
// src/lib/adMock.ts
export function showMockAd(): Promise<{ isEnded: boolean }> {
  return new Promise((resolve) => {
    const duration = 5000 // 模拟5秒广告
```

```
  // 弹出广告窗口、倒计时等 UI 逻辑
  setTimeout(() => resolve({ isEnded: true }), duration)
 })
}
```

8.2 后期对接抖音广告 SDK

上架抖音小游戏平台后，将 adMock 替换为 tt.createRewardedVideoAd 实现。广告位 ID 统一在 src/config/adConfig.ts 中配置，回调处理逻辑保持不变：监听 onClose 回调，若 res.isEnded=true 才调用后端确认接口，若 res.isEnded=false 仅提示用户，不调后端。

8.3 useAd Hook 接口设计

```
interface UseAdReturn {
 showAd: (adType: AdType) => Promise<{ isEnded: boolean }>;
 isAdLoading: boolean;
}
```

useAd Hook 统一封装广告调用逻辑。Demo 阶段内部调用 adMock.showMockAd()，上架后替换为 tt.createRewardedVideoAd 调用。外部调用接口保持不变，实现无感切换。

九、风控 UI 行为规则

前端根据后端返回的 userStatus 控制界面表现，规则如下：

userStatus	含义	前端行为
0（正常）	正常用户	所有功能正常开放
1（异常）	异常用户	增益入口隐藏或置灰；点击增益功能时弹出提示：“你有异常行为，暂定增益功能和积分奖励！”
2（非法）	非法用户	全屏显示“非法用户，拒绝登录！”，无法进入游戏

所有风控判定均在服务端 Edge Functions 中完成，前端无法绕过。前端只负责根据 userStatus 字段调整 UI 展示，不做任何本地风控判断。

十、附件：关键类型定义 (types.ts)

前端开发时需直接引入此类型文件，保证数据结构与后端接口一致。请勿手动修改字段名称和类型。以下是基于 Supabase 方案调整后的类型定义，主要变化是 openid 来源从抖音改为 Supabase Auth UUID，以及增加了 Supabase 相关的认证状态字段。

```
// types.ts
// 版本: V4.0 (Supabase 方案)
// 本文件由产品设计方提供，前端必须遵守。请勿手动修改字段名称和类型。

// ===================== 枚举 =====================
export enum UserStatus {
  NORMAL = 0,   // 正常
  ABNORMAL = 1, // 异常
  ILLEGAL = 2   // 非法
}

export enum IdentityLevel {
  SCHOLAR = 0,     // 书生
  TONGSHENG = 1,   // 童生
  XIUCAI = 2,      // 秀才
  JUREN = 3,       // 举人
  JINSHI = 4,      // 进士
  ZHUANGYUAN = 5,  // 状元
  TAIFU = 6,       // 太傅
  EMPEROR = 7      // 皇帝
}

export enum GameStatus {
  IDLE = 'IDLE',
  PLAYING = 'PLAYING',
  PAUSED = 'PAUSED',
  TIME_UP = 'TIME_UP',
  ANSWER_CORRECT = 'ANSWER_CORRECT',
```

```
  ANSWER_WRONG = 'ANSWER_WRONG',
  SESSION_OVER = 'SESSION_OVER',
  AD_PLAYING = 'AD_PLAYING',
  DAILY_OVER = 'DAILY_OVER',
}

export enum AdType {
  TIME_DOUBLE = 'time_double',
  REVIVE = 'revive',
}

export enum SessionResultFlag {
  PASS = 'A',        // 直接通过
  FAIL = 'F',        // 失败且未复活
  REVIVE = 'R',      // 复活（等待下一局）
  REVIVE_PASS = 'R->A',  // 复活后通过
  REVIVE_FAIL = 'R->F'   // 复活后失败
}

export enum PointsSource {
  ANSWER_REWARD = 'answer_reward',
  GIFT_EXCHANGE = 'gift_exchange',
  SIGN_IN = 'sign_in',
}

// ===================== 业务模型 =====================
export interface UserInfo {
  id: string;            // Supabase Auth UUID（替代原 number id）
  openid?: string;       // 抖音 openid（上架后填充）
  unionid?: string;
  nickname: string;
```

```
  avatarUrl: string;
  userStatus: UserStatus;
  identityLevel: IdentityLevel;
  totalPoints: number;
  currentMonthPoints: number;
  monthPassedDays: number;
  adContributionScore: number;  // 1-10
  mobile?: string;
  region?: string;
  hasUserInfoAuth: boolean;
  hasLocationAuth: boolean;
}

export interface Question {
  questionId: string;
  idioms: string[];
}

export interface SubmitAnswerRequest {
  sessionId: string;
  questionId: string;
  answer: string[];
}

export interface SubmitAnswerResponse {
  isCorrect: boolean;
  pointsEarned: number;
  correctOrder?: string[];
  totalAnswered: number;
  totalCorrect: number;
  isSessionComplete: boolean;
```

```
}

export interface SessionStartResponse {
  sessionId: string;
  sessionSeq: number;
  questions: Question[];
  initialTimeSeconds: number;
  canUseTimeDouble: boolean;
  timeDoubleRemaining: number;
  skipCountTotal: number;
  skipCountRemaining: number;
}

export interface SessionEndResponse {
  isSessionPassed: boolean;
  sessionPoints: number;
  correctCount: number;
  canRevive: boolean;
  reviveRemaining: number;
  dailyStatus: DailyStatus;
}

export interface DailyStatus {
  challengeDate: string;
  sessionResults: SessionResultFlag[];
  dailyPassed: boolean;
  dailyPoints: number;
  totalSessions: number;
  maxSessionsReached: boolean;
  needConsecutive: number;
  currentConsecutive: number;
```

```
}

export interface ReviveConfirmResponse {
  success: boolean;
  nextSessionStart: SessionStartResponse;
}

// ===================== 通用 =====================
export interface ApiResponse<T> {
  code: number;  // 0=成功，1=参数错误，2=未登录，3=权限不足，4=风控拦截，5=系统错误
  message: string;
  data: T;
}
```

十一、环境变量配置

前端项目需要配置以下环境变量，统一放在 .env.local 文件中：

变量名	示例值	说明
VITE_SUPABASE_URL	https://xxx.supabase.co	Supabase 项目 URL
VITE_SUPABASE_ANON_KEY	eyJhbGciOi...	Supabase Anon Key（公开安全密钥）

注意：.env.local 文件已加入 .gitignore，不得提交到 Git 仓库。团队成员需各自在本地创建该文件。可提供 .env.example 作为模板。

十二、交付物清单

- 完整项目源码：符合上述目录结构的 React 项目
- 可运行：npm install && npm run dev 能正常启动
- 可构建：npm run build 能生成生产包
- README：说明如何安装、运行、构建、配置 Supabase 环境变量
- .env.example：Supabase 环境变量模板

十三、验收清单

- 项目能正常安装依赖并启动
- src/types/types.ts 与本文档附件一致，无增删改字段
- 所有页面路由可正常跳转
- 游戏5个核心状态（IDLE/PLAYING/TIME_UP/ANSWER_CORRECT/ANSWER_WRONG/SESSION_OVER）的 UI 过渡正常
- 所有弹窗可正常弹出和关闭
- 广告 Hook 封装完整，Mock 广告流程可跑通
- 异常用户（userStatus=1）下，增益入口被隐藏或置灰
- Supabase Client 初始化正确，能成功调用 Edge Functions
- 代码有清晰注释，README 完整